Where Should an Error Point?

A line-placement study from the ErrorBuddy compiler: at the token where parsing failed, or at the line where the fix belongs

Vardan Pandya (24BCE2608)
Kunal Sharma (24BCE2641)

Abstract — A compiler error message normally points at the token where the parser found the problem. For most beginner mistakes this is also the line that has to be changed, but not for all of them: a missing semicolon is only noticed at the first token of the next statement, which can be a few lines further down. ErrorBuddy is a small compiler we wrote in C++17 for a beginner language, and its main aim is error messages that a beginner can act on. This case study looks at one design question from the project: which line should an error report? We made a second version of our compiler that reports the line where the parser stopped, and compared it with the normal version and with gcc 11.4 on 24 hand-labelled beginner mistakes. The normal version reported the correct line for all 24 mistakes, the second version for 21 and gcc for 20. gcc also gave no message at all for one division by zero. All of the wrong lines came from missing semicolons. We also tested the edit-distance limit used for the 'Did you mean' hints, which showed a bug in how our hints are chosen, and three ordering choices in the lexer. We show the output of the experiments as we ran them, explain what we found out from that output, show the compiler's real messages, and state the limits of the study.

Keywords: error messages, recursive descent parsing, syntax-directed translation, three-address code, edit distance, lexical analysis.

1. Scope and approach

Our README and presentation already describe the language, the compiler phases and the test programs, so this case study does not repeat them. It follows one thread: our claim that ErrorBuddy reports the line where the fix belongs. In the presentation this claim rested on one example, and we wanted to check it on more than one. Every number in this report comes from the script `case_study/run_study.py`. The script compiles our real C++ source files, and copies of them with small changes, and runs them on a set of test programs. So what we measure is the compiler itself and not a separate model of it.

Section 2 explains how the compiler got its current shape. Section 3 covers the lexer rules that every error message depends on. Sections 4 and 5 present the experiment and its results. Section 6 shows the output of the experiment exactly as it printed on our machine and explains what we found out from it. Section 7 draws the design conclusion, and Section 8 shows what ErrorBuddy outputs.

2. How we arrived at the idea

The starting point was a mistake that every beginner makes in C, a missing semicolon. For the program below gcc reports the error on line 5, although the mistake is on line 4. The message uses grammar words ('expected ... before') and does not say what to type.

```
4 | int b = 10
5 | printf("%d\n", a + b);

m01.c:5:1: error: expected ‘,’ or ‘;’ before ‘printf’
```

From this example we set three goals for every error message: (1) it must point to the line where the fix belongs, (2) it must use plain words, and (3) it must give one concrete fix. We then made the choices in Table 1 with these goals in mind.

Decision	What we chose	Reason	Cost
Language	int variables, + - * /, brackets, print, every statement ends with ;	Small enough that every phase and every message could be finished and tested	No if, while, for or input
Parser	Hand-written recursive descent, one function per grammar rule	When a rule fails we know which rule it was and which tokens are around it, so the message can be specific. A generated parser gives a generic 'syntax error'	Every error case is written by hand
Passes	Syntax, semantics and TAC in one pass	The parser already knows the context when a check fails	TAC is held back until the whole program has passed
Error object	CompileError {line, phase, message, suggestion}, used by every phase	Every error must have a line and a fix, and one function prints all of them	Compilation stops at the first error
Values	Each expression's value is computed while compiling	With no input and no loops every value is known, so a / zero can be proved to be a division by zero	Will need rework once input is added

Table 1. Main design decisions and what each one cost (from the README, the pitch and the source code).

The biggest change during the project was the language. Our proposal was for a subset of C with if, while, for and printf. While building it we cut this back to int variables, the four arithmetic operators, brackets and print, so that every phase and every error message could be finished and tested. Control flow is now future work. The second important choice was to compute the value of each expression while compiling. Since the language has no input and no loops, every value is known at compile time, and this is what lets ErrorBuddy prove that `a / zero` divides by zero when `zero` is 0.

3. The lexical groundwork: a fixed rule order and a keyword table

Every error message gives a line number and quotes tokens from the program, for example `Add a semicolon after '10'`. So the messages can only be right if the lexer produces the right tokens. Our lexer (`lexer.cpp`) is a hand-written scanner. At each position it tries the rules below in a fixed order. A word is first read completely and then looked up in a small keyword table; if it is not there, it stays an identifier.

```
RULES (tried in this order at every position):
  1 newline      \n                          line = line + 1
  2 whitespace   space, tab, \r
  3 comment      // up to the end of the line
  4 number       [0-9]+                      error if a letter follows, error if > 2147483647
  5 word         [A-Za-z_][A-Za-z_0-9]*      then keyword lookup
  6 symbol       + - * / = ( ) ;
    anything else                            error: unknown symbol
KEYWORDS = { int -> KW_INT, print -> KW_PRINT }
```

Experiment 3.1: what the rules produce. For the line `int bonus = (score - 5) / 5; // round down` the lexer emits KEYWORD(int) IDENTIFIER(bonus) ASSIGN LPAREN IDENTIFIER(score) OPERATOR(-) NUMBER(5) RPAREN OPERATOR(/) NUMBER(5) SEMICOLON. The comment produces no tokens, and each token keeps the line it came from.

Experiment 3.2: comment rule after the symbol rule. We moved the `//` check below the single-character symbols and compiled `int a = 10 / 2; // half of ten`. The first `/` was now taken as division, the comment became the tokens `/ / half of ten`, and the program failed with "A statement cannot start with '/'." The comment rule has to come first, because it is the longer match.

Experiment 3.3: no check for a digit followed by letters. We removed the check in rule 4 and compiled `int 2nd = 5;`. The lexer split the name into NUMBER(2) and IDENTIFIER(nd), and the parser then reported "Expected a variable name after 'int', but found '2'." With the check in place the error comes from the lexer and says what is actually wrong: "'2nd' is not a valid name. Variable names cannot start with a digit."

Experiment 3.4: keyword checked before the whole word is read. We matched `int` and `print` as prefixes before rule 5 and compiled `int interest = 5; int printer = 2;`. The variable interest was cut into the keyword int plus erest, and printer into print plus er, so the program failed with "'int' is a reserved keyword and cannot be used as a variable name." With the word read first and looked up afterwards, both names stay whole.

Why this matters for error messages. In 3.3 and 3.4 the changed lexer still gives an error, but the error is about something the student did not write. A wrong token boundary changes which explanation the student gets. Each token also stores its line number, which is how a message such as 'Error on line 2' can point to the right place.

4. The placement question and how we tested it

When our parser cannot continue, it is looking at one token, the lookahead. The simplest rule is to report the line of that token. We call this the *detection point*. Our parser does something different for some errors. In `expectSemicolon()` a missing semicolon is reported on `previous().line`, the line of the last token of the unfinished statement, and a missing `)` is also reported on the line of the token it should follow. We call this the *fix point*. The question is which of the two is right more often. Figure 1 shows where the line is chosen.

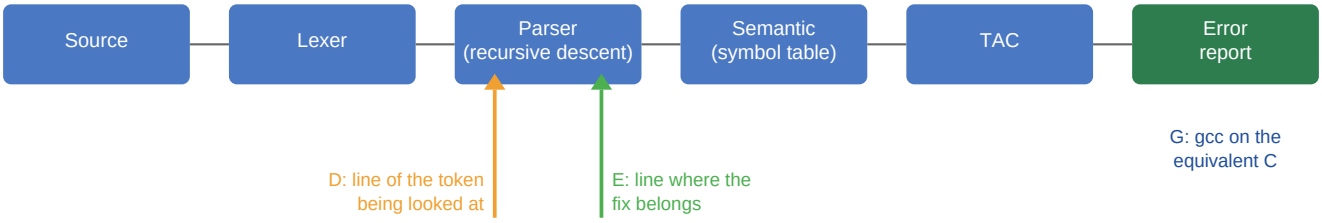

Figure 1. Where the reported line is chosen. D and E use the same lexer, parser, checks and messages and differ only in the line stored in CompileError. G is gcc on the same mistake written in C.

Variant	What runs	Which line it reports
G	gcc 11.4.0 with -std=c11 -Wall -Wextra -O2, on the same program written in C inside main()	The line of gcc's first error or warning
D	ErrorBuddy built from our source files with a 4-line change	Always the line of the lookahead token when the error was found (lexer errors keep their own line)
E	ErrorBuddy as submitted	The fix point, chosen for each error as described above

Table 2. The three variants.

Corpus and ground truth. We wrote 24 short programs (listed in Table 3), each with one mistake: missing semicolons in four positions, bracket and operator mistakes, undeclared and misspelled names, a duplicate declaration, a variable used in its own declaration, three kinds of division by zero, two overflows and two lexical mistakes. For each program the fix line, the line the student has to change, was decided by hand before running any variant. A report counts as correct only if its line equals the fix line. Each program was written twice, once in our language and once in C, where `print x;` becomes `printf("%d\n", x);` and the misspelled keyword becomes `prinft`. Mistakes that are legal C, such as a declaration without a value, were left out. The script is 232 lines of Python and the corpus 157 lines; both are supplied with this report so that every number below can be reproduced.

5. What happened

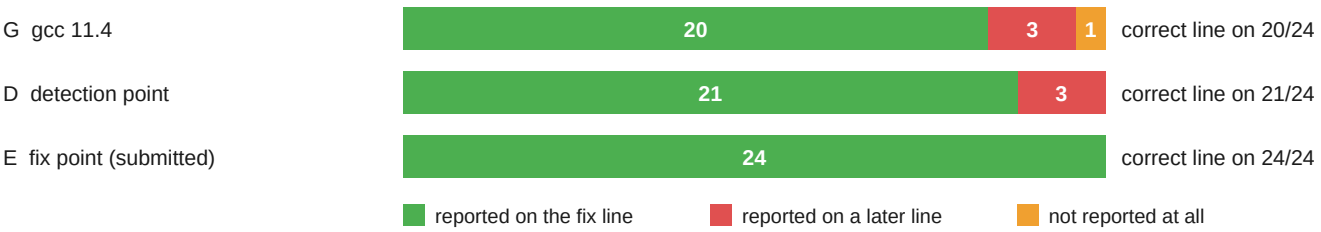

Figure 2. Summary of reported lines per variant on the 24-program corpus.

Program	Mistake	Fix line	G	D	E
M01	missing ; (next line is print)	2	3	3	2
M02	missing ; (blank line + comment after)	2	5	5	2

Program	Mistake	Fix line	G	D	E
M03	missing ; (next line is assignment)	2	3	3	2
M04	missing ; on last statement	3	3	3	3
M05	missing)	2	2	2	2
M06	missing) (expression split over lines)	3	3	3	3
M07	extra)	2	2	2	2
M08	two operators in a row	2	2	2	2
M09	operator with no right operand	2	2	2	2
M10	missing operator between values	3	3	3	3
M11	missing value after =	2	2	2	2
M12	empty parentheses	2	2	2	2
M13	undeclared variable	2	2	2	2
M14	misspelled variable	3	3	3	3
M15	misspelled print keyword	2	2 (warning)	2	2
M16	duplicate declaration	4	4	4	4
M17	variable used in its own declaration	1	1 (warning)	1	1
M18	division by literal 0	2	2 (warning)	2	2
M19	division by a variable that is 0	3	-	3	3
M20	division by an expression that is 0	1	1 (warning)	1	1
M21	arithmetic overflow	1	1 (warning)	1	1
M22	literal too large for int	1	1 (warning)	1	1
M23	unknown symbol	2	2	2	2
M24	name starts with a digit	1	1	1	1

Table 3. Reported line per program. Red: a later line than the fix line. Amber: no report. (warning): gcc only warned, so the C program still compiles.

5.1 The fix point (E) was exact

The submitted compiler reported all 24 mistakes on their fix lines. M06 was the case we expected to be difficult: the expression is split over two lines, the (is on line 2 and the missing) belongs on line 3. E reports line 3, and the message still names the line of the opening bracket: "Missing ')' to close the '(' opened on line 2."

5.2 The detection point (D) was late on missing semicolons

D and E differ on three programs only, M01, M02 and M03. All three are missing semicolons followed by another statement. The listing for M02 shows why:

```
1  int a = 5;
2  int b = 10              <- fix line: E reports line 2
3
4  // show the total
5  print a + b;            <- parser notices here: D and gcc report line 5
```

The parser cannot know that the semicolon is missing until it reaches print, the first token of the next statement. The blank line and the comment in between are skipped by the lexer, so the detection point ends up three lines below the mistake. M04, a missing semicolon on the last statement, is noticed at the end of the file, whose line is still the last line, so D gets it right by chance. Every other mistake in the corpus is noticed at the wrong token itself, so D and E agree on them.

5.3 gcc reported at the detection point

For all 23 mistakes that gcc reported, its line was the same as D's, including the three late ones. gcc gave a fix-it hint in five cases (M04, M05, M06, M14 and M15). It gave no message for M19, where the divisor is a variable that is 0 (`int zero = a - 10;` and then `a / zero`), even with -Wall -Wextra -O2. ErrorBuddy finds it because it keeps the value of every variable. Six of gcc's 23 reports were only warnings (M15, M17, M18, M20, M21 and M22), so the C program still compiles and a beginner can easily miss the message.

5.4 A second test: the limit for 'Did you mean' hints

For an undeclared name ErrorBuddy suggests a declared name with a small edit distance. The rule in `looksLike()` allows 1 edit for names of up to 3 letters and 2 edits for longer names, and names shorter than 3 letters are never suggested. We built three more versions that allow 1, 2 or 3 edits for every name and ran all four on 24 probes: 13 real typos of a declared name and 11 new names that should get no hint. The labels were decided before running.

Version	Rule	Right hint	Wrong hint	Missed	Correctly silent
k1	1 edit, all names	10	8	3	3
shipped	1 edit if <=3 letters else 2, skip names <3	12	6	1	5
k2	2 edits, all names	13	9	0	2
k3	3 edits, all names	13	10	0	1

Table 4. Hint results per version (submitted rule in green). 'Wrong hint' includes both a hint for a new name and a hint that names the wrong variable.

The submitted rule did best: 12 of 13 typos found, and no hint for one-letter names such as z and c, which every uniform rule got wrong. Any limit large enough to catch scroe (two letters changed) also suggests existing names for new names that are one letter away, such as dry and day or date and rate. The only typo the submitted rule missed is aeg for age, a swap of two neighbouring letters, which Levenshtein distance counts as 2 edits. The test also showed a bug. With first and last declared, the typo fast gets the hint 'first' (2 edits) instead of 'last' (1 edit). `similarVariable()` goes through the symbol table in alphabetical order and returns the first name within the limit, not the closest one.

```
~/ErrorBuddy$ python3 case_study/run_study.py --hints
typo    expect        k1 shipped      k2      k3
scroe   score          -   score   score   score
totl    total      total   total   total   total
cuont   count          -   count   count   count
avrage  average  average average average average
averge  average  average average average average
reslt   result    result  result  result  result
totals  total      total   total   total   total
numbr   number    number  number  number  number
conter  counter  counter counter counter counter
salry   salary    salary  salary  salary  salary
sun     sum          sum     sum     sum     sum
mx      max          max     max     max     max

typo    expect        k1 shipped      k2      k3
aeg     age            -       -     age     age
z       -              x       -       x       x
c       -              a       -       a       a
sum2    -            sum     sum     sum     sum
tax     -              -       -       -   total
store   -          score   score   score   score
height  -              -       -       -       -
dry     -            day     day     day     day
date    -           rate    rate    rate    rate
values  -          value   value   value   value
min     -              -       -     max     max
fast    -           last   first   first   first
```

Figure 4. Per-probe part of the hint test output (one listing, shown in two columns): the hint each version printed.

6. Output of the experiment and what we found out

After building the experiment we ran it on our own machine with `python3 case_study/run_study.py`. Figure 3 is the terminal output; the per-probe output of the hint test is Figure 4 in Section 5.4. This section reads that output line by line, because it is the evidence behind every claim in this report.

```
~/ErrorBuddy$ python3 case_study/run_study.py
case mistake                                    fix    G    D    E   gcc kind
M01  missing ; (next line is print)               2   3*   3*    2   error
M02  missing ; (blank line + comment after)       2   5*   5*    2   error
M03  missing ; (next line is assignment)          2   3*   3*    2   error
M04  missing ; on last statement                  3    3    3    3   error
M05  missing )                                    2    2    2    2   error
M06  missing ) (expression split over lines)      3    3    3    3   error
M07  extra )                                      2    2    2    2   error
M08  two operators in a row                       2    2    2    2   error
M09  operator with no right operand               2    2    2    2   error
M10  missing operator between values              3    3    3    3   error
M11  missing value after =                        2    2    2    2   error
M12  empty parentheses                            2    2    2    2   error
M13  undeclared variable                          2    2    2    2   error
M14  misspelled variable                          3    3    3    3   error
M15  misspelled print keyword                     2    2    2    2   warning
M16  duplicate declaration                        4    4    4    4   error
M17  variable used in its own declaration         1    1    1    1   warning
M18  division by literal 0                        2    2    2    2   warning
M19  division by a variable that is 0             3    -    3    3   none
M20  division by an expression that is 0          1    1    1    1   warning
M21  arithmetic overflow                          1    1    1    1   warning
M22  literal too large for int                    1    1    1    1   warning
M23  unknown symbol                               2    2    2    2   error
M24  name starts with a digit                     1    1    1    1   error

G  gcc 11.4 (first diagnostic)           caught=23/24  line-exact=20/24  late=3  early=0
D  ErrorBuddy, detection-point line      caught=24/24  line-exact=21/24  late=3  early=0
E  ErrorBuddy, fix-point line (shipped)  caught=24/24  line-exact=24/24  late=0  early=0
```

Figure 3. Output of run_study.py (Ubuntu 22.04, g++/gcc 11.4.0). The first block is the per-program table, the second the score of each variant.

6.1 How to read the output

The column **fix** is the fix line that we decided by hand before running anything. Columns **G**, **D** and **E** are the lines each variant reported. An asterisk means the line is not the fix line, and a dash means the variant gave no report. **gcc kind** says whether gcc's first message was an error or a warning. In the summary lines, **caught** is the number of mistakes reported at all, **line-exact** the number reported on the fix line, and **late** and **early** the number reported after or before it. In Figure 4 a dash means no hint was printed.

6.2 What we found out

#	Finding	Evidence in the output
1	**The same checks give different results with a different line rule.** D and E share every check and every message, but D was right on 21 programs and E on 24.	Summary lines: D line-exact=21/24, E line-exact=24/24.
2	**Only missing semicolons are affected.** The rows where D differs from the fix line are M01, M02 and M03. On the other 21 programs D and E print the same line.	Asterisks only in rows M01 to M03.
3	**The error can be more than one line away.** Blank lines and comments push the detection point further down.	M02: fix 2, G and D 5.
4	**gcc uses the detection point.** Wherever gcc reported, its line equals D's line.	Columns G and D are the same except M19.
5	**Keeping variable values finds more.** A divisor that is a variable holding 0 was not reported by gcc.	M19: G is a dash, gcc kind is none.
6	**Some real problems are only warnings in C.** In six cases gcc let the program compile.	gcc kind is warning on M15, M17, M18 and M20 to M22.
7	**No variant reported too early.** An early line would be worse than a late one, because the student would change correct code.	early=0 in all three summary lines.
8	**The hint chooses the first close name, not the closest.**	Figure 4, row fast: submitted version prints first.

Table 5. What the experiment output tells us.

End result in one paragraph. Reporting the line of the token where the parser stopped, which is what gcc does, was correct for 21 of 24 beginner mistakes. It was wrong for the most common one, the missing semicolon, by one to three lines. Choosing the line for each kind of error, as the submitted ErrorBuddy does, fixed all three cases

without changing any check or message. The hint limit we chose was the best of the four we tried, but the hint itself should be the closest name and not the first one found.

7. What this means for our design

Short answer. We keep the fix point for every error, and we will change the hint code to pick the closest name.

1. Keep choosing the line per error. This result has no downside. Three wrong lines were removed, no check or message changed, and no report got worse. The change in the code is small: `previous().line` instead of the lookahead's line in two places.

2. Write the rule down so that it carries over. The rule behind our choice is: if something is missing, report the line of the token it should follow; if something is wrong or extra, report the token itself. When we add if, while, for and braces, a missing } will be the same problem as a missing ; and this rule will apply to it.

3. Fix the hint ranking. similarVariable() should return the name with the smallest distance, and we could count a swap of two neighbouring letters as one edit so that aeg finds age. This is a few lines of code. We have not made the change yet, so all numbers in this report describe the submitted compiler.

4. Keep proven problems as errors. M17, M18 and M20 to M22 are only warnings in gcc, and M19 is not reported. In ErrorBuddy all six are errors. This is right for a language in which every value is known at compile time. Once input is added, 'the divisor is 0' and 'the divisor may be 0' will need different severities.

Threats to validity. (a) The corpus is small (24 programs and 24 hint probes) and written by us. The fix lines and hint labels were fixed before running, but some are judgement calls: for an undeclared name we take the line where it is used, and we labelled totals as a typo of total but values as a new name. (b) gcc compiles C and not our language. We wrote each mistake in both, but C's grammar is larger and this affects some of gcc's choices. (c) We only score gcc's first message and only its line; the comparison of wording in Section 8 is not measured. (d) Our language has no control flow, so mistakes with braces or else, where the line question is harder, are not tested. (e) We did not test with real beginners, so we assume, but have not measured, that the right line saves them time.

8. What ErrorBuddy outputs

Every error has the form *Error on line L*, a message, a *Suggestion* and the phase that found it. An error stops compilation with exit code 1, and no intermediate code is printed. A correct program prints its three-address code, its symbol table and the list of completed phases, with exit code 0. The outputs below were produced by the compiler.

Program	Output

`int a = 5;`
`int b = 10`
`print a + b;`
	`Error on line 2:`
	`Expected ';' at the end of the statement.`
	`Suggestion: Add a semicolon after '10'.`
	`(Found during: Syntax Analysis)`

`int x = 4;`
`int y = x + * 3;`
	`Error on line 2:`
	`Two operators in a row: '+' followed by '*'.`
	`Suggestion: Remove one of them, or put a number`
	`or variable between them.`

`int a = 10;`
`int zero = a - 10;`
`int b = a / zero;`
	`Error on line 3:`
	`Division by zero. Variable 'zero' is 0 at this point,`
	`so dividing by it is impossible.`
	`(Found during: Semantic Analysis)`

`int score = 10;`
`int bonus = 5;`
`scroe = score + bonus;`
	`Error on line 3:`
	`Variable 'scroe' has not been declared.`
	`Suggestion: Did you mean 'score'? If 'scroe' is a`
	`new variable, write 'int scroe = ...;'.`

`int 2nd = 5;`	`Error on line 1:`
	`'2nd' is not a valid name. Variable names cannot`
	`start with a digit.`
	`(Found during: Lexical Analysis)`

`int a = 5;`
`int b = 10;`
`int c = a + b * 2;`
`print c;`
	`Compilation successful!`
	`  a = 5`
	`  b = 10`
	`  t1 = b * 2`
	`  t2 = a + t1`
	`  c = t2`
	`  print c`

Table 6. Real ErrorBuddy output. Each error names the line, the problem, a fix and the phase.

The errors fall into three groups by phase. Lexical: unknown symbol, a name that starts with a digit, a number too large for an int. Syntax: missing ;, missing or extra), two operators in a row, an operator with nothing after it, a missing operator between two values, empty brackets, a missing value after =, negative literals, misspelled keywords and lines that are not statements. Semantic: undeclared or misspelled variable, duplicate declaration, a variable used in its own declaration, division by zero (by a number, a variable or an expression) and overflow. A correct program never produces a message.

9. Conclusions and lessons

(1) The line an error reports is a design choice and should not simply be the place where the parser stopped. In a 24-program experiment the same compiler went from 21 to 24 correct lines by changing only the line rule. (2) The detection point goes wrong for mistakes where something is missing, because these are only noticed at the next token. (3) A compiler that knows every value can prove problems that gcc only warns about or does not see. (4) Hints need a ranking as well as a limit; our own test found that the submitted hint can name the wrong variable. (5) Small lexer choices (the comment rule first, reading the whole word before the keyword lookup) decide which explanation the student finally reads.

Reproducing the experiment. Run `make test` (10 of 10 examples pass), then `python3 case_study/run_study.py`. It prints the table and summary lines shown in Figure 3. The option `--hints` prints Table 4 and Figure 4, and `--lexer` runs the experiments of Section 3. The corpus with its fix lines is in `case_study/corpus.py`.